A statistical comparison of human written and AI-generated code across programming languages

Akanksha Gupta (25BAI0033)

Bhargavi Panwar (25BDS0081)

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India

Bhargavi.panwar.2025@vitstudent.ac.in

Akanksha.gupta2025@vitstudent.ac.in

Abstract

The development of artificial intelligence has given rise to new opportunities. The structure of the programs developed by AI and humans might be dissimilar. This paper provides a statistical comparison of code created by humans and AI in C, C++, Java and Python. The dataset used in this paper comprises 10,000 samples of software code. Half of it is human written and the other half is AI. Characteristics such as lines of code, number of characters, normalized character density, comments and number of comments are taken into account. Statistical methods such as descriptive analysis, Welch's t-test, confidence intervals, Holm correction, Cohen's d, Cliff's delta, Wilcoxon rank-sum test, Spearman correlation coefficient, regression analysis, and factorial analysis of variance are applied. The results show that differences between two groups are not consistent among all the languages. The final analysis further found higher median characters per nonblank line and higher comment presence in AI samples across all four languages. The differences in raw code length varied by language, and the results describe patterns in this dataset rather than proof of authorship or code quality.

Introduction

In today's time AI is on a high rise. Along with it so are AI assisted programming tools. They have become increasingly relevant is software development. Code can be generated now just by small basic prompts. This can be a good or a bad thing. While it does help increase speed and efficiency, on the other hand it causes the individual to be overly dependent on AI. It can also cause vulnerabilities and security risks. For such reasons it is important we

differentiate between what's AI and what's human. For this purpose, we will be examining the structure and characteristics of both AI and human code.

AI has had considerable impact on software development. It's the result of the rise of large language models that can generate codes from natural language prompts. Tools like ChatGPT, Claude, etc have facilitated programming by enabling programmers to easily create code eliminate redundant work. However, the increasing use of such tools has led to concerns regarding the reliability, consistency and nature of the generated code.

Hence, it becomes necessary to compare the AI and human codes. The code produced by humans could be different based on the experience of the programmer, their coding style, and their knowledge.

An approach based on analysing characteristics of the source code might provide a fresh outlook since it compares statistical features of both types of code. The features could include the number of lines, number of characters, character density, and comments. These characteristics would not be consistent across all programming languages as each language has unique features and structures.

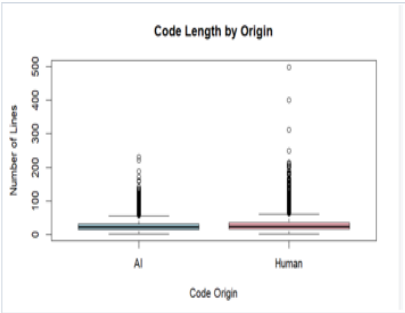
The current study thus analyses the statistical differences in the codes produced by humans and AI technology for C, C++, Java, and Python. This involves the application of statistics on source codes as measurements to explore the differences in length, density, and the use of comments. This research also involves analysis of the association between the number of lines and number of characters used in the codes.

Dataset and Methodology

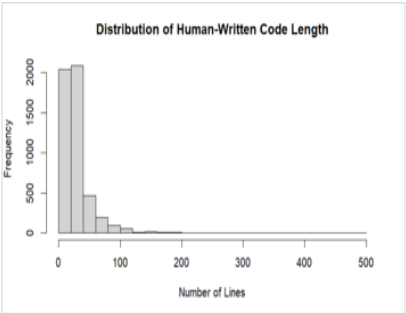
A. Data Methodology

The dataset comprises 10,000 observations including both human and AI code samples 5,000 observations each. The samples are written in C, C++, Java, and Python. This dataset was gathered through the work conducted by Boukili et al. (2026). This source includes the labelled examples of both human and AI-generated code. Observations include the information on the programming language, code origin, problem identifier, and source-code content.

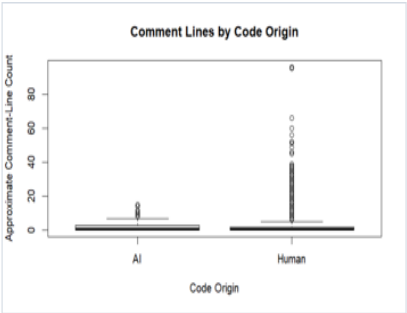
It is worth noting that the number of samples is not equal among the four programming languages. Thus, there are 1,737 codes C, 2,640 codes in C++ language, 2,945 codes in Java and 2,678 codes in Python language. The balanced distribution between human and AI codes give an equal comparison size. However, this is the construction of the dataset and not the proportion of in general AI written code.



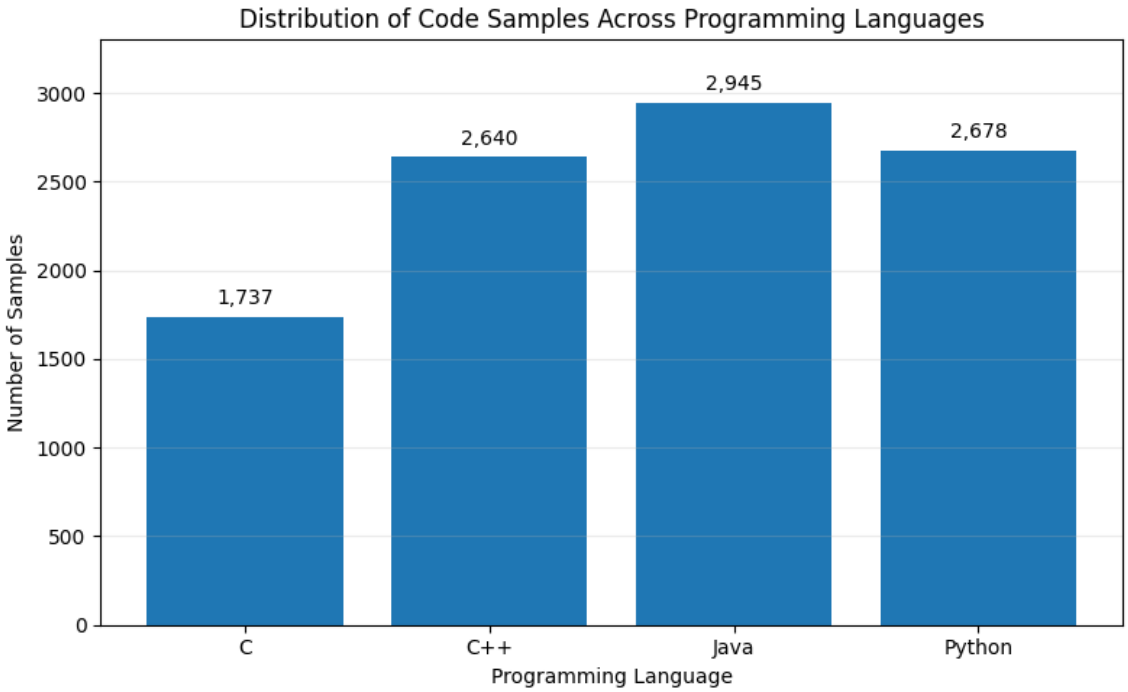
Code length: medians are close yes, 21 vs 23 and both the groups have long right tails (basically more code lengths are shorter with a few high number).



Human code length is strongly right skewed means a few very long programs pull the mean up.



Human comment counts have a long tail we see maximum 96 vs 15 for AI.



B. Data Preprocessing and Feature Extraction

Source code content was analysed to get characteristics that can be used for further statistical analysis. Firstly, we have used line count character count for measure. They are used to get basic data about the size of each program. Line count is defined as the number of lines in the code sample. The character count is defined as the number of characters stored in this sample. Thus, one can make comparisons of the size of code written by humans and AI.

Character density has also been analysed to make some additional measures. It is calculated by dividing the number of characters on nonblank lines by the total number of nonblank lines.

Comment features have been studied using two different measures: comment presence and comment volume. Comment presence refers to the existence of any comment line in the sample code. Comment volume is the number of comment lines detected. For comment recognition, language specific

conventions have been used and for Python the hash symbol #, while for C, C++ and Java the comment markers are // and /*.

C. Statistical Methods

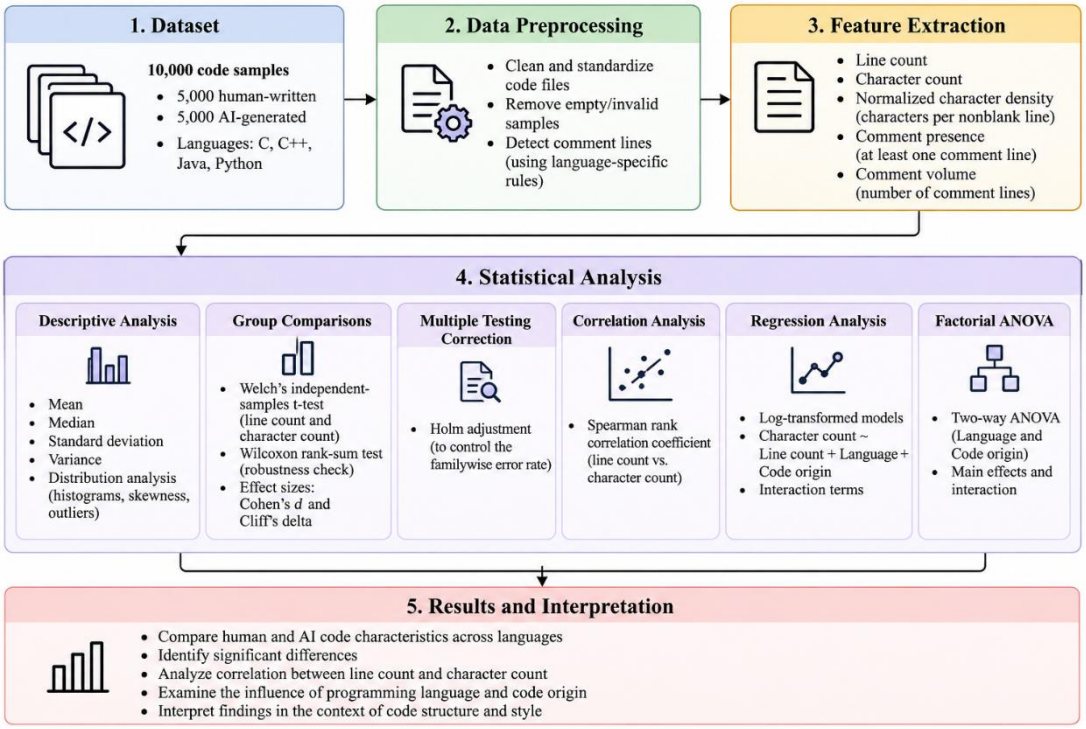
Descriptive statistical parameters such as mean, median, standard deviation, and variance were estimated to provide insight into the dataset. They allowed us to understand how code length and other derived variables varied.

Histograms and boxplots were also used to understand the shape of the measurements. The distributions showed right-skewness and extreme values. For comment counts, the variance-to-mean ratios ranged from 2.05 to 28.34 across the language and origin groups, indicating overdispersion compared with a simple Poisson reference.

The Welch's independent t-test was run to assess whether there were significant differences between mean line number and character number for human and AI programs. Here, each programming language was tested independently.

Because several statistical tests had been conducted, the Holm adjustment was made to control increasing number of false positives. Cohen's d and Cliff's delta were considered to evaluate the size of differences between groups and to provide additional information about them. The Wilcoxon rank-sum test is a non-parametric robustness test for comparing the distributions violating the assumptions of the parametric tests.

Spearman rank correlation coefficient was calculated to test the relationship between line count and character count. Then regression analysis was conducted to analyse the connection between the code length, programming language and code origin. Finally, factorial analysis of variance was considered to analyse the separate influence of these variables and their interaction on the data obtained.

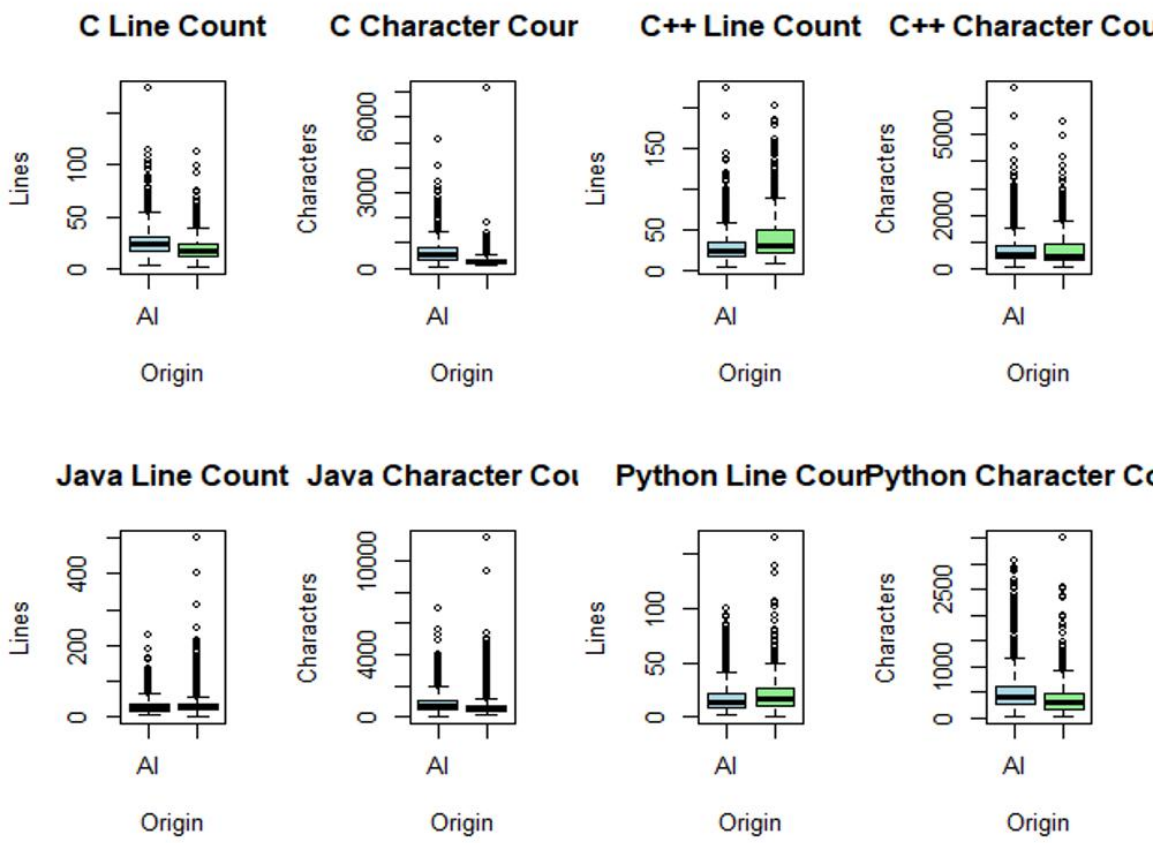


Results and Analysis

A. Descriptive Analysis of Code Length

The descriptive analysis examined the differences in line count and character count between human and AI generated programs. The initial comparison indicated the differences varied across programming languages. In C, AI generated samples exhibited higher mean line counts and character counts than human samples. In C++, human-written programs had a higher mean line count, while the mean character counts of the two groups were relatively similar.

A different pattern was observed in Java and Python. Human samples had higher mean line counts in both languages, whereas AI generated samples exhibited higher mean character counts. These observations suggest that the relationship between code origin and code length is not consistent across programming languages. Therefore, analysing the dataset as a single combined group may not be efficient.



B. Hypothesis Testing and Effect Analysis

In order to find out if there is any statistical difference in the observed group mean, Welch's test for independent samples has been run on line count and character count individually for each language. Holm correction has also been used in order to make sure that there will be no false positive results.

The results of the test analysis revealed that seven out of eight language specific differences were statistically significant after Holm correction. The character count for C++ was proved to not be statistically significant.

Effect size measurements were also considered because statistical significance alone does not indicate the practical magnitude of a difference. Cohen's d provided a standardised measure of the difference between group means. Cliff's delta offered a rank-based measure of distributional separation. The inclusion of these measures allowed the observed differences to be interpreted in terms of their magnitude rather than relying on their p values.

The Wilcoxon rank sum test was performed as an additional robustness check. It provides a complementary perspective when the underlying distributions are skewed or contain extreme observations.

A chi-square test of independence was also conducted for code origin and programming language. The result was significant, $\chi^2(3) = 87.519$, $p < 2.2 \times 10^{-16}$, showing an association between the origin labels and language distribution in this dataset. Since the dataset was constructed with equal numbers of AI and human samples, this result should not be interpreted as real-world usage or as a causal relationship.

For C++ character count, the Welch test was not significant (Holm-adjusted p = 0.536), while the Wilcoxon test was significant (Holm-adjusted p = 0.000908). However, Cliff's delta was only 0.075, showing that the rank-based difference was small.

Welch test results-

The confidence intervals are the Welch 95% intervals for each comparison. Holm correction was applied to the p-values across the eight tests.

Languag e	Measurement	AI − Human mean	95% CI of difference	Holm-adjusted p	Conclusion
C	Line count	+7.10	[5.74, 8.45]	2.37×10^{-23}	Significant
C	Character count	+359.60	[322.46, 396.75]	5.47×10^{-72}	Significant
C++	Line count	−9.16	[−10.90, −7.42]	1.03×10^{-23}	Significant
C++	Character count	+13.70	[−29.71, 57.11]	0.5361	Not significant
Java	Line count	−4.49	[−6.41, −2.58]	1.34×10^{-5}	Significant
Java	Character count	+149.15	[100.95, 197.35]	5.89×10^{-9}	Significant
Python	Line count	−1.79	[−2.91, −0.67]	0.003562	Significant
Python	Character count	+143.78	[116.71, 170.86]	4.59×10^{-24}	Significant

The mean-based and rank-based effect sizes are shown below. Positive values indicate higher AI measurements, while negative values indicate higher human measurements.

Language	Measurement	Cohen's d	Cliff's delta
C	Line count	0.476	0.315
C	Character count	0.873	0.687
C++	Line count	−0.395	−0.248
C++	Character count	0.024	0.075
Java	Line count	−0.164	−0.106
Java	Character count	0.223	0.246
Python	Line count	−0.122	−0.087
Python	Character count	0.392	0.271

C. Normalized Character Density

Raw character count does not indicate the distribution of characters across lines. The study investigated this through the calculation of character density which was done by dividing the number of characters on non-blank lines by the number of non-blank lines.

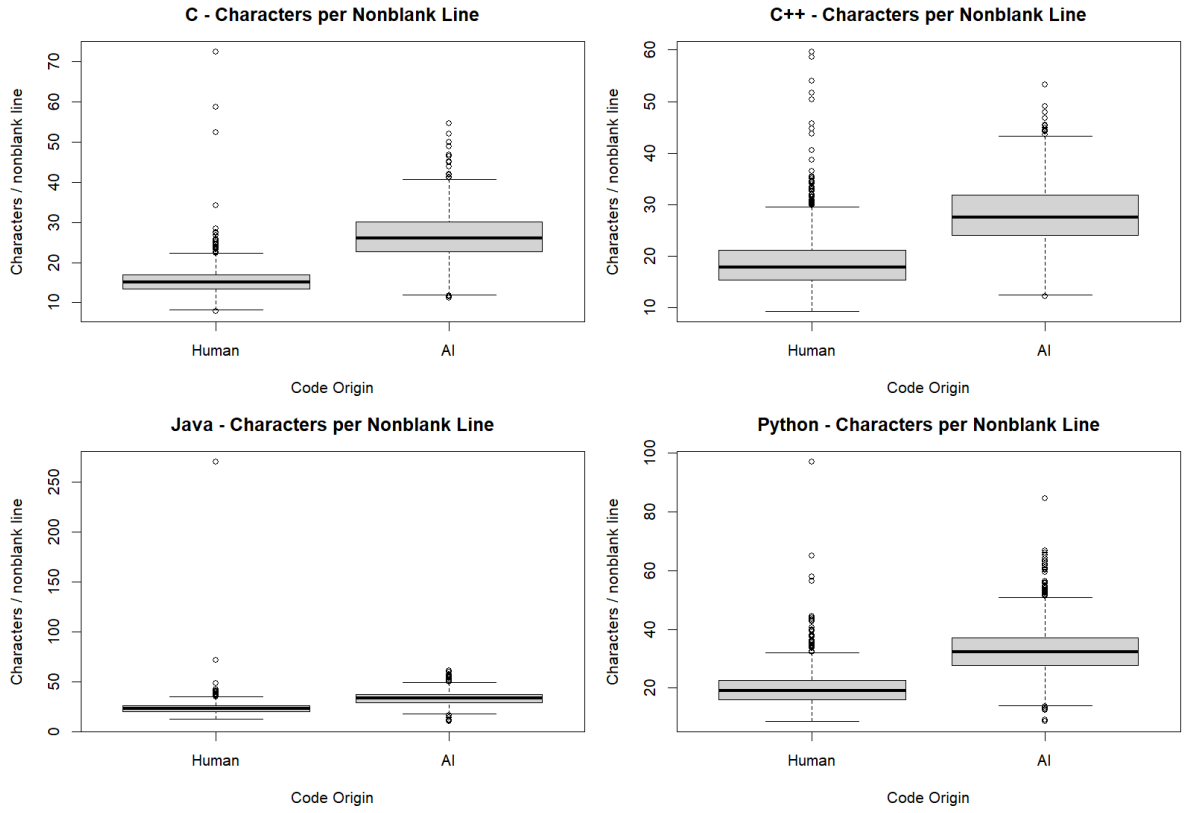
The findings revealed that samples of codes generated by AI had higher median character density than human samples across all four languages. The difference was particularly pronounced in C where the median character density was 26.1 for AI while human had a density of 15.1. The same trend was seen in Python

where median density of AI was 32.2, while that of humans was 19.1. This indicates that codes generated by AI had more characters per line in the data set.

This finding provides information other than raw program length. It reveals that differences in the two groups can also be related to the distribution of code on individual lines.

The language-wise median values and rank-based effect sizes are provided below. All four comparisons remained significant after Holm correction.

Language	AI median	Human median	Median difference	Cliff's delta	Holm-adjusted p
C	26.12	15.14	+10.98	0.921	$< 2.2 \times 10^{-16}$
C++	27.50	17.89	+9.61	0.806	$< 2.2 \times 10^{-16}$
Java	33.22	22.82	+10.40	0.805	$< 2.2 \times 10^{-16}$
Python	32.17	19.08	+13.08	0.855	$< 2.2 \times 10^{-16}$



D. Comment Characteristics

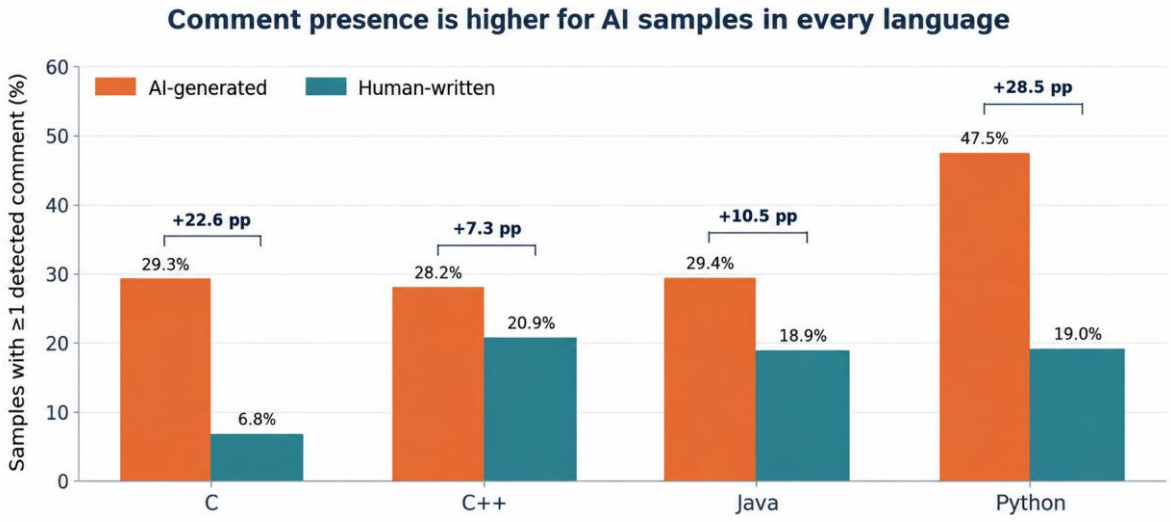
Characteristics of comments were analysed in order to explore whether there is any difference between human and Ai written comments. Firstly, it should be noted that the analysis made a difference between the presence of at least one detectable comment line and the number of comment lines.

It turned out that in all programming languages, the proportion of comment presence was higher among the samples written by AI. The biggest difference

was seen in the samples written in Python. The gap between the two categories is 28.5 percentage points. Hence, it can be said that the commenting characteristic can be another difference between the two groups in the chosen dataset.

The differences in comment presence were also tested as differences in proportions. All four language-wise comparisons remained significant after Holm correction. The largest difference was in Python (28.48 percentage points), followed by C (22.58 points), Java (10.52 points) and C++ (7.31 points).

Comment volume showed a different pattern from comment presence. The median detected comment count was zero in both groups in every language. The mean count was higher for AI in C and Python, while it was higher for human samples in C++ and was almost the same in Java. This is why comment presence and comment volume were treated as separate measures.
However, the comment analysis needs to be discussed with care. There was a possibility that not all comments were detected using text-based rules. Comment markers within strings, inline comments and some language-specific comment forms could distort results. Therefore, the analysis shows only approximate commenting characteristics.
The comment-count distributions also showed overdispersion, with variance-to-mean ratios from 2.05 to 28.34 across the language and origin groups. This suggests that a simple Poisson model would not describe the comment counts well. A stratified sample of 80 code records was prepared for manual checking of the comment counter, but the manual labels have not been completed, so detection accuracy is not reported.

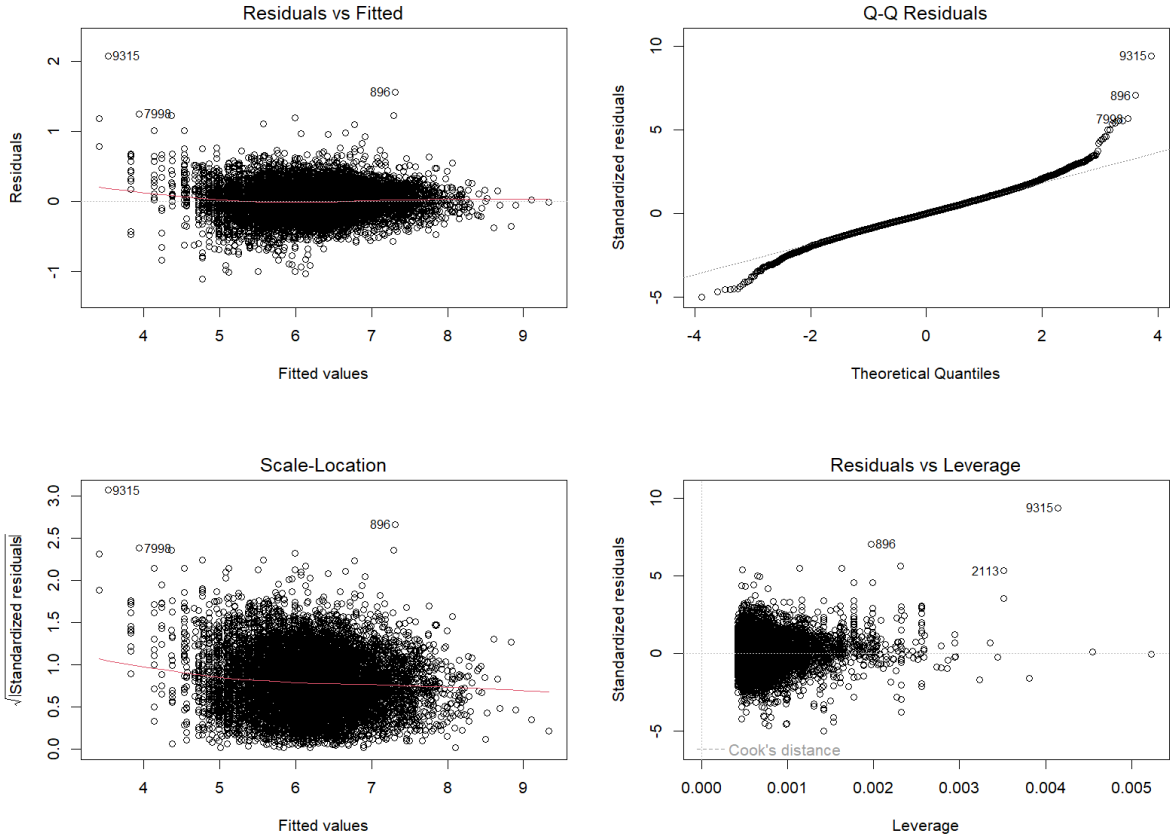


E. Correlation Analysis

Correlation between line count and character count was evaluated using Spearman's rank correlation coefficient. This method was used because it was necessary to evaluate the strength and direction of the association between the given variables, without assuming their linear relationship.

Overall, there is a strong positive association between line count and character count with Spearman correlation coefficient 0.882 and p value less than $2.2e^{-16}$. There are positive associations when the correlation is calculated separately for the four different programming languages. It means that programs with more lines usually have more characters as well.

The obtained correlation results will be a good starting point for further regression analysis.



F. Regression Analysis

Regression analysis was conducted using log-transformed character count as the outcome, with log-transformed line count, programming language and code origin as predictors. Adding the language-by-origin interaction improved the

model fit compared with the additive model, $F(3, 9991) = 64.17$, $p = 4.34 \times 10^{-41}$. Adding a line-count-by-origin interaction also improved the fit, $F(1, 9990) = 10.08$, $p = 0.00151$. These results suggest that the relationships in the data vary across origin and language.

The diagnostic plots showed departures from normality in the residual tails and possible unequal residual variance. Therefore, the regression results are interpreted as exploratory associations and not as a validated prediction model.

G. Factorial Analysis of Variance

Analysis of variance using the factorial method (ANOVA) was applied to test the impact of programming language and code source on the selected characteristics. The use of the factorial ANOVA makes it different from other methods where separate comparisons were made in each language.

The interaction of programming language and code source is significant to this. Testing the interaction can help find out statistically whether the effect of code source varies depending on the programming language.

Thus, the analysis is supplementary to the comparison of code in each language. The results confirm the necessity to analyse language dependent differences in measurable characteristics of human AI written source code.

The updated factorial analysis found significant effects of language and origin, as well as significant language-by-origin interactions, for both outcomes. The table reports the F statistics, partial eta-squared values and Holm-adjusted p-values for all six effects.

	A	B	C	D	E	F	G	H
1	Outcome	Term	DF_effect	DF_error	F_value	P_value	Partial_Eta	Holm_P_within_outcome
2	Line_Coun	Language	3	9995	532.0924	7.7617712	0.137714	2.32853138884979e-320
3	Line_Coun	Origin adju	1	9995	47.94856	4.64E-12	0.004774	4.64E-12
4	Line_Coun	Language	3	9992	69.59595	1.54E-44	0.020468	3.09E-44
5	Character_	Language	3	9995	601.6139	0	0.152955	0
6	Character_	Origin adju	1	9995	544.5147	########	0.051664	5.18E-117
7	Character	Language	3	9992	105.183	4.84E-67	0.030613	4.84E-67

Discussion

The study results show that the observable differences in human and AI generated programming code cannot be characterized by a unified pattern. The comparison of the line count and characters used shows that the differences depend on the programming language. While in some comparisons the AI generated sample had the highest character mean. The comparison of character counts for C++ did not reveal the statistically significant

difference according to Welch's t test. Therefore, it is necessary to analyse programming languages separately, rather than relying on similar characteristics of all languages.

Normalized character density helped to compare the distribution of characters per nonblank lines. AI generated samples demonstrated higher median values in all four languages. This shows that there are differences in the structure of code representation regardless of the overall size of the program. Likewise, the comments comparison showed that the AI generated samples were characterized by the higher likelihood of the presence of comments. However, the study results describe the observable patterns in the selected data set. They do not prove that one group generates better code.

Most importantly, the experiment concentrates on structural features and not the program's correctness, efficiency, security or its general quality. Differences observed in the two groups through statistical analysis thus cannot be used to prove authorship or identify a particular program as having been created by AI.

Another limitation is that the human-written and AI-generated samples were collected through different processes. Some observed differences may therefore also reflect source, task or generation conditions, and the analysis cannot isolate a causal effect of code origin. The problem-ID check found only three IDs represented in both origin groups. This describes the observed overlap but does not prove that all observations are independent.

The comment counter was not fully validated through manual annotation. Although an 80-record stratified validation sample was prepared, the labels were not completed for this report. The comment findings should therefore be treated as approximate.

Conclusion

This study presented a statistical comparison of human and AI generated code across C, C++, Java and Python using a dataset of 10,000 code samples. The analysis examined line count, character count, normalized character density and commenting characteristics through a combination of descriptive and statistical techniques. The results demonstrated that differences between the two groups vary across programming languages. This makes language specific analysis important for interpreting code characteristics.

The character density and comment presence analysis revealed additional patterns beyond code length. AI generated samples showed higher median character density and a greater

likelihood of containing detectable comments across the four programming languages. Correlation and regression analyses further highlighted the relationship between line count and character count. Factorial ANOVA provided a framework for examining the combined effects of programming language and code origin.

In the corrected normalized measure, the median characters per nonblank line were higher for AI samples in C (26.12 versus 15.14), C++ (27.50 versus 17.89), Java (33.22 versus 22.82) and Python (32.17 versus 19.08). Comment presence was also higher for AI samples in each language, with the largest gap in Python at 28.48 percentage points. These findings provide the main additional insight of the final analysis, while remaining limited to the selected dataset and the measurement rules used.

Overall, the study demonstrates how statistical methods can be applied to codes to investigate differences between human AI generated programs. Although the findings identify measurable patterns within the selected dataset, they should not be generalised to all programming environments. Future work may extend the analysis using larger and more diverse datasets, improved comment-detection methods and statistical models.

References

[1] G. Boukili, S. El Garouani, and J. Riffi, “A dataset for human-written and AI-generated code source classification,” Data in Brief, vol. 65, Art. no. 112527, 2026.

[2] B. Demirok, M. Kutlu, and S. Mergen, “MultiAIGCD: A comprehensive dataset for AI generated code detection covering multiple languages, models, prompts, and scenarios,” Neural Computing and Applications, vol. 38, Art. no. 687, 2026.

[3] A. Gurioli, M. Gabbrielli, and S. Zacchiroli, “Is This You, LLM? Recognizing AI-written Programs with Multilingual Code Stylometry,” arXiv:2412.14611, 2024.

[4] B. Demirok and M. Kutlu, “AIGCodeSet: A New Annotated Dataset for AI Generated Code Detection,” arXiv:2412.16594, 2024.